Deep Work & Brainstorming Technical Design

Workflow Engine & Form Builder untuk Probis Collection

Item	Keterangan
Dokumen	Hasil riset dan rancangan teknis sebagai bahan deep work, diskusi SA/Tech Lead, SRS, dan roadmap MVP.
Tema	Probis Collection, Workflow / State Engine, Form Builder
Tanggal penyusunan	7 Juni 2026
Basis internal	Summary Detail dan MoM meeting Probis Collection - Workflow Engine - Form Builder, 4 Juni 2026.
Stack awal	Vue.js 3.5.35, Node.js 24.16.0, MongoDB 8.3.2, MinIO, PostgreSQL 16
Sudut pandang	Professional System Analyst: scope, architecture, data model, workflow logic, form metadata, APIs, code idea, risk, dan delivery plan.

Executive position: Rekomendasi utama: jadikan Probis Collection sebagai quick win yang paling jelas, lalu implementasikan Workflow Engine dan Form Builder sebagai metadata-driven MVP. Jangan menjanjikan full BPMN execution engine atau low-code platform enterprise dalam fase 3 bulan.

Daftar Isi

- 1. Executive Summary
- 2. Konteks Bisnis dan Prinsip Solusi
- 3. Scope MVP dan Non-Goals
- 4. Arsitektur Solusi yang Direkomendasikan
- 5. Desain Probis Collection
- 6. Desain BPMN Compatibility dan Parser
- 7. Desain Workflow / State Engine
- 8. Desain Form Builder Metadata-Driven
- 9. Data Architecture: PostgreSQL, MongoDB, dan MinIO
- 10. API Design dan Contract Awal
- 11. Bottleneck Analytics dan Reporting
- 12. Security, Governance, dan Auditability
- 13. Evaluasi Tech Stack
- 14. Ide Implementasi Code
- 15. Roadmap 3 Bulan
- 16. Testing Strategy
- 17. Risk Register
- 18. Keputusan Teknis yang Perlu Dikunci
- 19. Kesimpulan dan Next Action
- 20. Referensi

1. Executive Summary

Kebutuhan project ini sebaiknya dipahami sebagai metadata-driven business process platform, bukan sebagai full low-code platform dan bukan sebagai full BPMN execution engine. Scope meeting menekankan bahwa sistem perlu mendukung collection proses bisnis lintas entitas, workflow tracking, approval/reject/return, form dinamis, dan analisis bottleneck. Namun timeline 3 bulan membuat desain perlu dipotong menjadi MVP yang jelas dan dapat didemokan bertahap [M1][M2].

Rancangan yang paling aman adalah memisahkan domain menjadi empat layer: Probis Collection, BPMN/Process Model, Workflow State Engine, dan Form Builder, lalu membungkusnya dalam Application Shell yang mengatur login, role, entity context, dashboard, dan admin master data. Pembagian layer ini konsisten dengan arah meeting bahwa Probis Collection adalah fondasi, Workflow Engine tidak wajib product penuh, BPMN dapat menjadi visual/reference, dan Form Builder dibuat generated form yang rapi dan usable [M1][M2].

Secara teknis, Node.js + TypeScript layak menjadi backend utama atau minimal service khusus untuk parser BPMN, workflow, dan form metadata karena ekosistem JavaScript mendukung bpmn-js, bpmn-moddle, JSON Forms/FormKit, dan state machine library. Node.js 24 tercatat sebagai LTS di situs resmi Node.js per 2026, sehingga lebih cocok daripada release current/odd-numbered untuk deployment enterprise [R1][R2].

Untuk data, rekomendasi kuatnya adalah PostgreSQL 16 menjadi source of truth untuk workflow runtime, audit event, assignment, RBAC, dan reporting; MongoDB 8.3 digunakan untuk form template dan form submission dinamis bila approval teknologi mengizinkan; MinIO digunakan untuk menyimpan BPMN XML, attachment form, bukti approval, dan dokumen terkait. Jika organisasi tidak ingin mengoperasikan dua database, fallback paling rasional adalah PostgreSQL JSONB untuk form metadata dan submission karena PostgreSQL mendukung jsonb, indexing, dan operator JSON yang matang [R6].

MVP perlu didefinisikan dengan batasan eksplisit: support import/render BPMN, extract basic step/sequence flow, bind form ke state, run custom state transition, record audit trail, dan tampilkan bottleneck sederhana. Out-of-scope fase awal: executable BPMN penuh, drag-and-drop form designer kompleks, runtime expression engine bebas, parallel gateway kompleks, compensation, timer event, multi-instance activity, dan generated UI pixel-perfect.

Keputusan Rekomendasi	Alasan SA	Dampak
Gunakan custom state machine, bukan full BPMN engine pada MVP.	Requirement utama adalah tracking, approval, reject/return, audit, dan perhitungan durasi; full BPMN execution akan menaikkan kompleksitas drastis.	Lebih cepat dideliver, lebih mudah dikontrol, dan tetap kompatibel dengan BPMN sebagai visual/metadata.
PostgreSQL sebagai source of truth workflow.	Workflow membutuhkan transaksi, locking, audit, join reporting, dan konsistensi assignment.	Auditability dan query bottleneck lebih kuat.
MongoDB hanya untuk form dinamis atau fallback ke PostgreSQL JSONB.	Form metadata/submission berubah-ubah; MongoDB fleksibel, tetapi multi-database menambah operasional.	Ada dua opsi sesuai approval teknologi.
Versioning wajib untuk Probis, workflow definition, BPMN artifact, dan form template.	Instance yang sudah berjalan tidak boleh berubah diam-diam saat template/form diubah.	Mencegah konflik audit dan bug historis.
MVP 3 bulan fokus ke 1-2 skenario end-to-end.	Stakeholder perlu melihat progres, tetapi scope harus terkendali.	Demo per sprint dan acceptance criteria menjadi jelas.

2. Konteks Bisnis dan Prinsip Solusi

Berdasarkan dokumen meeting, sistem ditujukan untuk membantu MDG/IT Business Partner melakukan assessment terhadap kebutuhan aplikasi atau inisiatif digital: apakah kebutuhan tersebut align dengan core function dan proses bisnis entitas terkait. Probis Collection menjadi bank data proses bisnis lintas divisi/unit/entitas, sedangkan workflow dan form dipakai untuk menjalankan, mencatat, dan menilai progres proses yang berbasis tahapan [M1][M2].

Modul	Prinsip	Penjelasan	Konsekuensi Desain
Form Generator & Metadata Management	Metadata-first	Proses, state, transition, form, dan assignment didefinisikan sebagai metadata.	Developer tidak perlu hardcode setiap variasi proses, tetapi tetap ada batasan type dan validation.
Workflow Engine	Versioned configuration	Setiap Probis, BPMN, workflow, dan form memiliki versi draft/published/archived.	Instance runtime selalu menunjuk ke versi yang dipublish saat instance dibuat.
Workflow Engine	Audit append-only	Perubahan status tidak menimpa history lama.	Workflow event menjadi bukti proses dan sumber analytics.
Workflow Engine	BPMN compatible, not BPMN complete	BPMN dipakai sebagai notasi visual, import, dan metadata extraction.	Subset BPMN dieksekusi melalui state machine; elemen kompleks diperlakukan

Modul	Prinsip	Penjelasan	Konsekuensi Desain
			sebagai manual mapping.
Form Generator & Metadata Management	Form usable, not pixel-perfect	Generated form cukup rapi, konsisten, dan seperti Google Form-like.	UI final custom per entitas menjadi fase lanjutan.
Process Business Repository	Entity context (Entitas Organisasi)	User bekerja dalam konteks organisasi/entitas/role.	Semua list, task, form, dan report harus difilter berdasarkan akses.

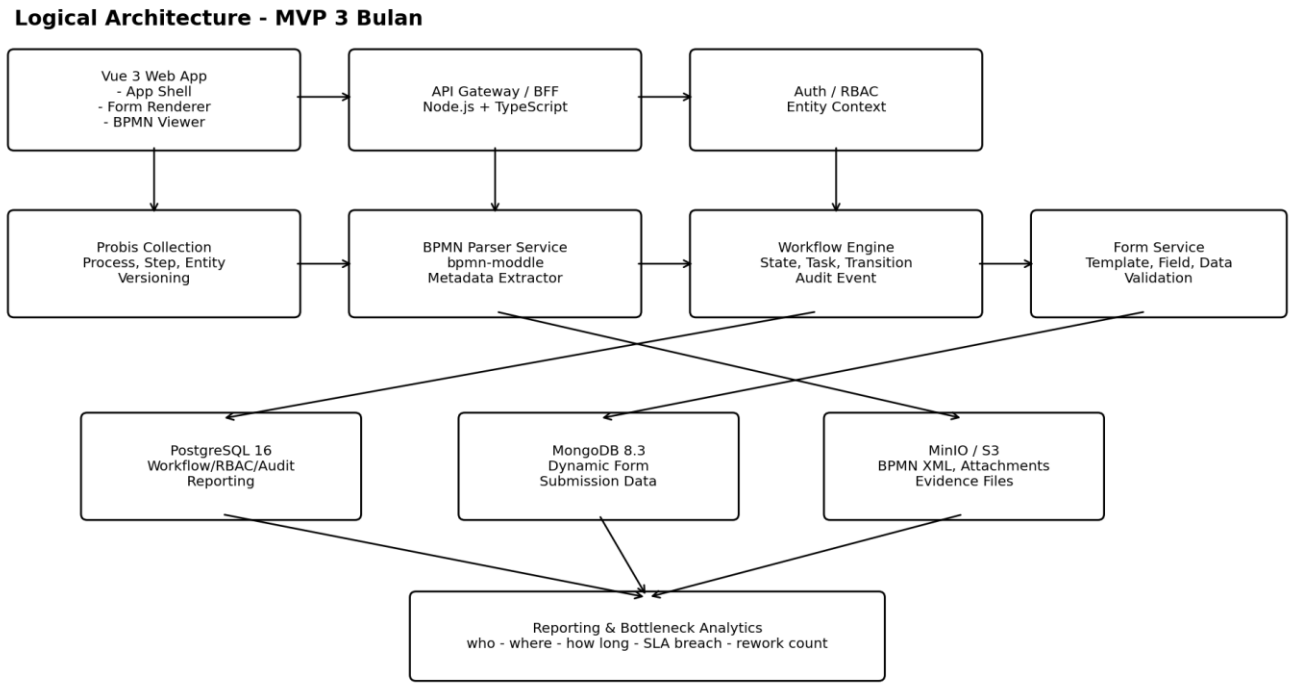
3. Scope MVP dan Non-Goals

Batasan scope adalah kontrol utama agar delivery 3 bulan realistis. Dokumen meeting sudah menekankan perlunya membedakan quick win dengan platform SAP-like/low-code enterprise [M1][M2].

Kategori	Fitur	Status MVP
Probis Collection	CRUD Probis, entity mapping, tahapan, subflow sederhana, tag, owner, status versi.	Must
BPMN	Upload/import BPMN XML, render diagram, simpan artifact, extract task dan sequence flow dasar.	Must
Workflow	State definition, transition approve/submit, reject, return previous/originator, assignment, task list, audit trail.	Must
Form Builder	Form template sederhana: text, textarea, number, date, select, radio, checkbox, file, section, required, basic validation.	Must
Form Mapping	Map 1 form ke 1 user task; state boleh tanpa form.	Must
Reporting	Dashboard status instance, aging task, SLA breach, bottleneck by state/role/user.	Must/Should
Security	RBAC, entity-based access, audit, file upload restriction, presigned object access (Object Storage/MinIO). Batasan: Di Role atau User Group	Must
Advanced BPMN	Parallel gateway, event gateway, timer/boundary event, compensation, multi-instance, complex subprocess execution.	Won't MVP
Advanced Form Designer	Drag-and-drop builder lengkap, conditional logic kompleks, custom layout pixel-perfect, formula bebas.	Won't MVP
AI features	AI-generated workflow/form, auto recommendation process, NLP to BPMN.	Won't MVP

4. Arsitektur Solusi yang Direkomendasikan

Arsitektur disarankan berbentuk modular monolith Node.js + TypeScript terlebih dahulu, bukan microservice penuh. Untuk timeline 3 bulan, modular monolith memberi pemisahan domain tetap jelas tanpa overhead deployment, observability, dan distributed transaction yang terlalu besar. Bila organisasi menuntut pemisahan service, domain yang paling aman dipisah adalah BPMN Parser Service dan Form Service.



Gambar 1. Logical architecture MVP: Vue sebagai web app, Node.js API/BFF, PostgreSQL untuk workflow/audit/reporting, MongoDB untuk form dinamis, dan MinIO untuk object storage.

Modul	Tanggung Jawab	Catatan Implementasi
Application Shell	Login, menu, active entity, user profile, dashboard shell, notification.	Jangan dicampur dengan domain workflow agar akses dan audit konsisten.
Probis Collection	Kelola proses bisnis, step, subflow, entity mapping, ownership, versioning.	Fondasi knowledge base dan referensi workflow definition.
BPMN Parser/Viewer	Render BPMN, parse XML, extract element dasar, validate subset MVP.	Frontend memakai bpmn-js; backend parse dengan bpmn-moddle.
Workflow Engine	Run instance, transition validation, task assignment, audit event, SLA, bottleneck metrics.	Source of truth di PostgreSQL.
Form Builder/Renderer	Form template, field metadata, validation, render Vue, submission data.	MongoDB atau PostgreSQL JSONB; wajib snapshot version.
Reporting	Status, aging, SLA breach, bottleneck, rework count.	Query dari event/task history.
Object Storage	BPMN XML, attachment form, evidence approval, exports.	Gunakan MinIO/S3 presigned URLs dan metadata references.

Architecture recommendation: Untuk quick win, hindari microservice jika belum ada platform observability dan DevOps matang. Buat package/module terpisah di repo yang sama: modules/probis, modules/workflow, modules/forms, modules/bpmn, modules/reporting. Domain boundaries tetap jelas, deployment tetap sederhana.

5. Desain Probis Collection

Probis Collection adalah knowledge base proses bisnis. Tujuannya bukan hanya menyimpan diagram, tetapi membentuk traceability antara entitas, fungsi bisnis, proses, step, subflow, dan potensi reuse lintas entitas. Dokumen meeting menyebut traceability dua arah sebagai kebutuhan penting [M1].

Object	Deskripsi	Field Kunci
Organization / Entity	Divisi, unit induk, direktorat, stream, atau fungsi bisnis.	id, code, name, parent_id, type, status
Probis	Proses bisnis utama yang menjadi collection.	id, code, name, description, owner_entity_id, status, version
Probis Step	Tahapan atomik yang dapat dipakai sebagai referensi workflow state.	id, probis_id, step_key, name, order_no, actor_role, description
Subflow	Rangkaian step reusable lintas proses/entity.	id, code, name, version, reusable_flag
Entity-Probis Mapping	Relasi entitas dengan proses yang dimiliki/diikuti.	entity_id, probis_id, involvement_type, role

Object	Deskripsi	Field Kunci
BPMN Artifact	File BPMN XML atau diagram referensi.	id, probis_id, version, object_key, checksum, parsed_status
Process Version	Snapshot definisi probis dan step pada saat publish.	id, probis_id, version_no, published_at, published_by

5.1 Lifecycle Probis

1. Draft: BA/SA membuat atau mengubah Probis, step, entity mapping, dan BPMN artifact.
2. Review: owner proses atau stakeholder bisnis meninjau nama step, aktor, dan batasan proses.
3. Published: versi Probis dapat dipakai untuk workflow definition dan form binding.
4. Archived: versi lama tidak dipakai untuk instance baru, tetapi tetap dibaca oleh instance historis.

6. Desain BPMN Compatibility dan Parser

BPMN tetap penting karena merupakan notasi yang dimengerti stakeholder dan menjadi representasi visual proses. OMG menjelaskan BPMN sebagai standar de-facto untuk diagram proses bisnis yang cukup mudah dipahami stakeholder, tetapi cukup presisi untuk diterjemahkan ke software process components [R9]. Dalam MVP, posisinya adalah compatible artifact dan metadata source, bukan mesin eksekusi penuh.

Kemampuan BPMN	MVP	Keterangan
Render BPMN diagram	Ya	Gunakan bpmn-js di frontend untuk viewer/modeler browser [R10].
Upload/import BPMN XML	Ya	Simpan raw XML di MinIO dan metadata di PostgreSQL.
Parse task dan sequence flow	Ya	Gunakan bpmn-moddle di backend untuk read/write BPMN 2.0 XML [R11].
Lane/pool to role mapping	Should	Lane dapat menjadi kandidat actor/role, tetapi perlu validasi manual.
Exclusive gateway sederhana	Should	Gate condition sebaiknya didefinisikan sebagai transition guard metadata.
Parallel gateway penuh	Optional / Phase 2	Membutuhkan join/split semantics dan task concurrency.
Boundary/timer/event/compensation	Out of scope	Menaikkan kompleksitas engine dan test coverage.
Round-trip edit BPMN lengkap	Out of scope MVP	MVP cukup import/render; editing visual bisa fase berikutnya.

Contoh code idea - BPMN metadata extraction

```
// Concept: parse BPMN XML and extract simple elements for process metadata.
import { BpmnModdle } from 'bpmn-moddle';

type ExtractedStep = {
  bpmnId: string;
  type: string;
  name: string;
  incoming: string[];
  outgoing: string[];
};

export async function extractProcess(xml: string) {
  const moddle = new BpmnModdle();
  const { rootElement: definitions } = await moddle.fromXML(xml);

  const process = definitions.rootElements.find(
    (el: any) => el.$type === 'bpmn:Process'
  );
  if (!process) throw new Error('BPMN process not found');

  const flowElements = process.flowElements ?? [];
  const supportedTypes = new Set([
    'bpmn:StartEvent', 'bpmn:EndEvent', 'bpmn:Task', 'bpmn:UserTask',
    'bpmn:ManualTask', 'bpmn:ServiceTask', 'bpmn:ExclusiveGateway'
  ]);

  const steps: ExtractedStep[] = flowElements
    .filter((el: any) => supportedTypes.has(el.$type))
    .map((el: any) => ({
      bpmnId: el.id,
```

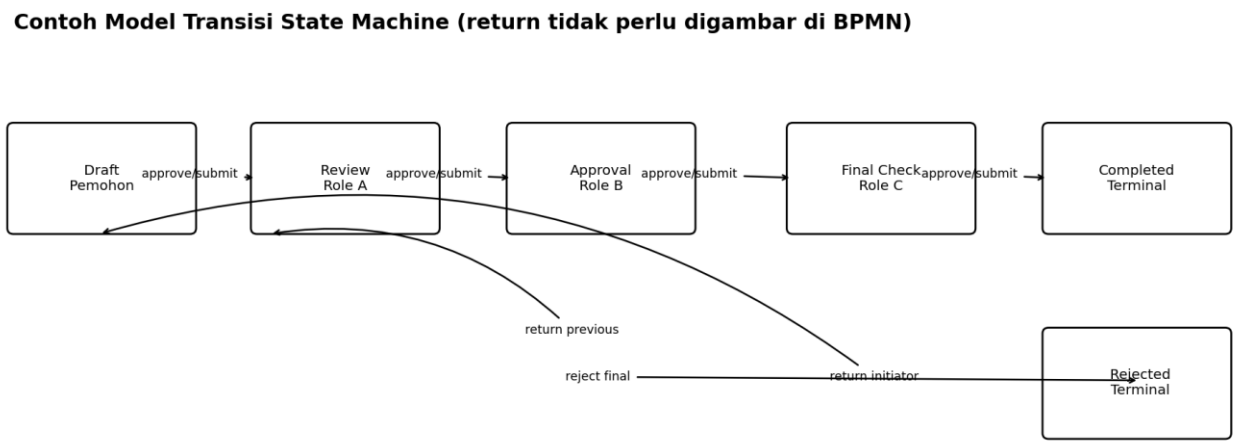
```
      type: el.$type,
      name: el.name ?? el.id,
      incoming: (el.incoming ?? []).map((x: any) => x.id),
      outgoing: (el.outgoing ?? []).map((x: any) => x.id),
    }));

  const flows = flowElements
    .filter((el: any) => el.$type === 'bpmn:SequenceFlow')
    .map((el: any) => ({
      id: el.id,
      name: el.name,
      sourceRef: el.sourceRef?.id,
      targetRef: el.targetRef?.id,
    }));

  return { processId: process.id, processName: process.name, steps, flows };
}
```

7. Desain Workflow / State Engine

Workflow Engine dalam konteks project ini adalah state engine yang menjalankan instance proses berdasarkan definition metadata. Requirement minimalnya: mencatat posisi proses, actor/task, approval, reject/return, history, dan durasi untuk bottleneck analysis. XState dapat menjadi referensi/library karena mendukung state machines/statecharts dan actor model untuk JavaScript/TypeScript, tetapi MVP juga dapat dibuat custom agar persistence dan audit lebih sesuai kebutuhan enterprise [R14].



Gambar 2. Return/reject diperlakukan sebagai transition rule di state engine, bukan harus selalu digambar sebagai backward edge di BPMN.

Konsep	Definisi	Contoh
Workflow Definition	Template workflow versi publish.	Pengajuan Initiative v1
State	Tahap proses yang dapat memiliki assignment dan form.	draft, review_mdg, approval_sdi, completed
Transition	Aksi legal dari state asal ke state tujuan.	submit, approve, return_previous, reject_final
Guard	Syarat sebelum transition boleh dijalankan.	form_complete, user_has_role, amount <= limit
Action	Efek samping setelah transition.	create task, send notification, set SLA timer
Task	Work item untuk user/role tertentu.	Review oleh Role A
Event/Audit	Catatan immutable setiap aksi.	STATE_ENTERED, TASK_COMPLETED, TRANSITION_APPROVED

7.1 Transition Rules

Transition Type	Behavior	Audit yang Dicatat
SUBMIT / APPROVE	Pindah ke next state sesuai definition.	actor, from_state, to_state, action,

Transition Type	Behavior	Audit yang Dicatat
		comment, timestamp
RETURN_PREVIOUS	Kembali ke state sebelumnya berdasarkan event path atau previous_state_key.	returned_by, return_reason, returned_to_state, target_task
RETURN_TO_INITIATOR	Kembali ke correction state/pemohon awal.	returned_by, initiator_user_id, reason
RETURN_TO_SPECIFIC_STATE	Kembali ke state tertentu yang diizinkan oleh rule.	target_state_key, rule_id, reason
REJECT_FINAL	Masuk terminal rejected; tidak membuat next task.	rejected_by, reason, final_status
REASSIGN	Tidak mengubah state, hanya memindahkan task owner.	old_assignee, new_assignee, reason

Contoh code idea - transaction-safe workflow transition

```
// Concept: transition command should be transaction-safe and audit-first.
export async function executeTransition(command: {
  instanceId: string;
  action: 'SUBMIT' | 'APPROVE' | 'RETURN_PREVIOUS' | 'RETURN_TO_INITIATOR' | 'REJECT_FINAL';
  actorUserId: string;
  taskId?: string;
  comment?: string;
  formSubmissionId?: string;
}) {
  return db.transaction(async (trx) => {
    const instance = await trx.workflowInstance.findForUpdate(command.instanceId);
    const definition = await trx.workflowDefinition.getPublished(instance.definitionVersionId);

    const currentState = definition.states[instance.currentStateKey];
    const transition = resolveTransition(definition, currentState, command.action);

    await assertTaskIsActionable(trx, command.taskId, command.actorUserId);
    await assertUserHasRole(command.actorUserId, transition.allowedRoles);
    await assertGuardsPassed({ trx, instance, transition, command });

    const toStateKey = await resolveTargetState({
      trx, instance, transition, action: command.action,
    });

    await trx.workflowTask.complete(command.taskId, command.actorUserId);
    await trx.workflowInstance.updateState(instance.id, toStateKey, instance.version + 1);

    await trx.workflowEvent.insert({
      instanceId: instance.id,
      eventType: 'TRANSITION_EXECUTED',
      fromStateKey: instance.currentStateKey,
      toStateKey,
      action: command.action,
      actorUserId: command.actorUserId,
      comment: command.comment,
      formSubmissionId: command.formSubmissionId,
      occurredAt: new Date(),
    });

    await createNextTasksIfNeeded(trx, definition, instance.id, toStateKey);
    return trx.workflowInstance.get(instance.id);
  });
}
```

8. Desain Form Builder Metadata-Driven

Form Builder versi MVP sebaiknya tidak diposisikan sebagai drag-and-drop low-code designer penuh. Inti yang diperlukan adalah form metadata: daftar field, label, type, required, option, validation, order, grouping, dan mapping ke state/entity. JSON Forms menyediakan pendekatan form berbasis JSON Schema dengan dukungan Vue [R12], sedangkan FormKit memiliki schema JSON-serializable untuk menyimpan DOM structure dan form generation [R13]. Untuk quick win, pilih salah satu dari dua pendekatan: adopsi library schema-driven, atau buat custom metadata yang lebih sederhana dan mudah dikontrol.

Aspek	Rekomendasi MVP	Catatan
Form Template	Versioned JSON metadata.	Setiap perubahan publish menghasilkan versi baru.
Field Types	text, textarea, number, date, datetime, select, radio, checkbox, file, section.	Hindari nested repeating table di MVP kecuali wajib.
Validation	required, min/max, regex, enum, file type/size.	Gunakan validator backend sebagai source of truth.
Conditional	VisibleIf sederhana berdasarkan field lain.	Jangan izinkan expression JavaScript bebas dari database.
Binding	State dapat 0 form, 1 form, atau multiple forms.	Mapping disimpan di workflow_state_from_binding.
Submission	Simpan payload + template_version_id + snapshot metadata.	Agar data historis tetap bisa dibaca walau template berubah.
UI/UX	Google Form-like: section, label jelas, help text, error message, progress.	Bukan pixel-perfect custom form.

Contoh metadata form

```
// Simplified form template metadata stored in MongoDB or PostgreSQL JSONB.
{
  "code": "initiative_assessment_form",
  "version": 3,
  "status": "published",
  "title": "Form Assessment Inisiatif Digital",
  "fields": [
    {
      "key": "initiativeName",
      "type": "text",
      "label": "Nama Inisiatif",
      "required": true,
      "maxLength": 200,
      "order": 10
    },
    {
      "key": "businessEntity",
      "type": "select",
      "label": "Entitas Bisnis",
      "required": true,
      "optionsSource": "master:organization",
      "order": 20
    },
    {
      "key": "urgency",
      "type": "radio",
      "label": "Urgensi",
      "required": true,
      "options": ["Low", "Medium", "High"],
      "order": 30
    },
    {
      "key": "supportingDocument",
      "type": "file",
      "label": "Dokumen Pendukung",
      "required": false,
      "acceptedMimeTypes": ["application/pdf"],
      "maxSizeMb": 10,
      "order": 40
    }
  ]
}
```

Contoh code idea - Vue dynamic form renderer

```
<!-- Concept: dynamic Vue 3 renderer. Use component mapping, not v-html. -->
<script setup lang="ts">
type Field = { key: string; type: string; label: string; required?: boolean; options?: string[] };
const props = defineProps<{ fields: Field[]; modelValue: Record<string, unknown> }>();
const emit = defineEmits<{ (e: 'update:modelValue', value: Record<string, unknown>): void }>();
```

```
function update(key: string, value: unknown) {
  emit('update:modelValue', { ...props.modelValue, [key]: value });
}

function componentFor(type: string) {
  return {
    text: 'TextInput', textarea: 'TextareaInput', number: 'NumberInput',
    date: 'DateInput', select: 'SelectInput', radio: 'RadioGroupInput',
    checkbox: 'CheckboxInput', file: 'FileInput'
  }[type] ?? 'UnsupportedField';
}
</script>

<template>
  <div class="dynamic-form">
    <component
      v-for="field in fields"
      :key="field.key"
      :is="componentFor(field.type)"
      :field="field"
      :model-value="modelValue[field.key]"
      @update:model-value="value => update(field.key, value)"
    />
  </div>
</template>
```

9. Data Architecture: PostgreSQL, MongoDB, dan MinIO

Data architecture harus menjaga dua hal: fleksibilitas metadata dan auditability. PostgreSQL 16 cocok untuk workflow runtime karena JSONB tersedia bila perlu fleksibilitas tambahan, dan dokumentasi PostgreSQL menyebut jsonb lebih cepat diproses daripada json karena disimpan dalam format binary decomposed serta mendukung indexing [R6]. MongoDB 8.3 dapat digunakan untuk dokumen form yang dinamis; dokumentasi MongoDB mencatat 8.3-series sebagai current stable release, dan minor releases dinyatakan stable serta suitable untuk production workloads [R3][R4].

Storage	Data yang Disimpan	Alasan
PostgreSQL 16	Workflow definition/runtime, task, event audit, RBAC, organization, probis metadata, reporting materialized view.	Transaksi, locking, relational integrity, reporting SQL, audit query.
MongoDB 8.3	Form template, form submission, dynamic payload, optional draft autosave.	Schema fleksibel untuk field dinamis, dokumen dapat mengikuti form version.
MinIO / S3	BPMN XML, attachment, exported report, evidence file.	Object storage lebih cocok untuk binary/file daripada database.
PostgreSQL JSONB fallback	Form template/submission bila MongoDB tidak disetujui.	Mengurangi komponen operasional sambil tetap mendukung JSON dan index.

9.1 Relational Table Draft

DDL draft - workflow core

```
-- Simplified PostgreSQL table draft. Naming can be adjusted to project convention.
CREATE TABLE organization (
  id uuid PRIMARY KEY,
  code varchar(50) UNIQUE NOT NULL,
  name varchar(255) NOT NULL,
  parent_id uuid NULL REFERENCES organization(id),
  org_type varchar(50) NOT NULL,
  is_active boolean NOT NULL DEFAULT true
);

CREATE TABLE probis (
  id uuid PRIMARY KEY,
  code varchar(80) UNIQUE NOT NULL,
  name varchar(255) NOT NULL,
  owner_org_id uuid REFERENCES organization(id),
```

```
  description text,
  status varchar(30) NOT NULL DEFAULT 'draft',
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE workflow_definition (
  id uuid PRIMARY KEY,
  probis_id uuid NOT NULL REFERENCES probis(id),
  version_no int NOT NULL,
  status varchar(30) NOT NULL, -- draft/published/archived
  definition_json jsonb NOT NULL,
  published_at timestamptz,
  UNIQUE (probis_id, version_no)
);

CREATE TABLE workflow_instance (
  id uuid PRIMARY KEY,
  definition_id uuid NOT NULL REFERENCES workflow_definition(id),
  business_key varchar(120) NOT NULL,
  initiator_user_id uuid NOT NULL,
  current_state_key varchar(120) NOT NULL,
  status varchar(30) NOT NULL,
  version int NOT NULL DEFAULT 1,
  started_at timestamptz NOT NULL DEFAULT now(),
  completed_at timestamptz
);

CREATE TABLE workflow_task (
  id uuid PRIMARY KEY,
  instance_id uuid NOT NULL REFERENCES workflow_instance(id),
  state_key varchar(120) NOT NULL,
  assigned_role varchar(120),
  assigned_user_id uuid,
  status varchar(30) NOT NULL, -- open/completed/cancelled
  created_at timestamptz NOT NULL DEFAULT now(),
  claimed_at timestamptz,
  completed_at timestamptz
);

CREATE TABLE workflow_event (
  id uuid PRIMARY KEY,
  instance_id uuid NOT NULL REFERENCES workflow_instance(id),
  task_id uuid NULL REFERENCES workflow_task(id),
  event_type varchar(80) NOT NULL,
  from_state_key varchar(120),
  to_state_key varchar(120),
  actor_user_id uuid NOT NULL,
  comment text,
  event_data jsonb,
  occurred_at timestamptz NOT NULL DEFAULT now()
);
```

MongoDB collection draft - form templates and submissions

```
// MongoDB collection idea: form_templates
{
  _id: ObjectId("..."),
  code: "initiative_assessment_form",
  version: 3,
  status: "published",
  title: "Form Assessment Inisiatif Digital",
  fields: [ ... ],
  createdBy: "uuid-user",
  createdAt: ISODate("2026-06-07T00:00:00Z"),
  publishedAt: ISODate("2026-06-07T00:00:00Z")
}
```

```
// MongoDB collection idea: form_submissions
{
  _id: ObjectId("..."),
  workflowInstanceId: "uuid-instance",
  workflowTaskId: "uuid-task",
  stateKey: "review_mdg",
  formCode: "initiative_assessment_form",
  formVersion: 3,
  templateSnapshot: { title: "...", fields: [ ... ] },
  data: {
    initiativeName: "...",
    businessEntity: "DIV-ABC",
    urgency: "High"
  },
  attachments: [
    { bucket: "workflow-attachments", objectKey: "2026/06/...pdf", sha256: "..." }
  ],
  submittedBy: "uuid-user",
  submittedAt: ISODate("2026-06-07T00:00:00Z")
}
```

10. API Design dan Contract Awal

Area	Endpoint	Deskripsi
Probis	GET /api/probis	List/search Probis berdasarkan entity context dan permission.
Probis	POST /api/probis	Create Probis draft.
BPMN	POST /api/probis/{id}/bpmn/import	Upload BPMN XML, simpan artifact, parse metadata.
BPMN	GET /api/probis/{id}/bpmn/{version}	Ambil BPMN XML atau signed object URL.
Workflow Definition	POST /api/workflow-definitions	Create draft definition dari Probis/BPMN metadata.
Workflow Definition	POST /api/workflow-definitions/{id}/publish	Publish definition version.
Workflow Instance	POST /api/workflow-instances	Start workflow instance.
Task	GET /api/tasks/my	Task list untuk user berdasarkan role/entity.
Task	POST /api/tasks/{id}/transition	Approve/return/reject task.
Form	POST /api/form-templates	Create/update form template draft.
Form	POST /api/form-submissions	Submit data form untuk task/state.
Report	GET /api/reports/bottlenecks	Bottleneck metrics by process/state/role/user.

API contract example - transition task

```
POST /api/tasks/{taskId}/transition
Content-Type: application/json

{
  "action": "APPROVE",
  "comment": "Data sudah sesuai",
  "formSubmissionId": "665fb1c...",
  "idempotencyKey": "b2a4b88e-4c9e-4b7a-9e8b-..."
}

Response 200
{
  "instanceId": "uuid-instance",
  "previousState": "review_mdg",
  "currentState": "approval_sdi",
  "status": "in_progress",
  "nextTasks": [
    { "taskId": "uuid-task", "stateKey": "approval_sdi", "assignedRole": "SDI_APPROVER" }
  ]
}
```

11. Bottleneck Analytics dan Reporting

Requirement menyebut kemampuan analisis bottleneck: siapa, di mana, dan berapa lama. Ini sebaiknya tidak dihitung dari current_state saja, tetapi dari event/task history yang append-only. Minimal analytics yang perlu dibuat dalam MVP: aging task open, average duration per state, p95 duration, SLA breach count, return/rework count, dan ranking actor/role yang paling lama memproses task.

Metric	Formula / Source	Tujuan
State duration	STATE_ENTERED to STATE_EXITED atau task.created_at to task.completed_at.	Melihat tahap yang paling lama.
Queue time	task.created_at to task.claimed_at atau first action.	Melihat task menunggu siapa.
Work time	task.claimed_at to completed_at.	Membedakan idle vs active processing.
SLA breach	duration_hours > sla_hours per state.	Prioritas follow-up.
Return count	Count TRANSITION_RETURNED per instance/state.	Melihat proses yang sering rework.
Actor bottleneck	Avg/p95 completed task duration by role/user.	Menjawab siapa yang menjadi bottleneck.

SQL idea - bottleneck by state and role

```
-- Bottleneck query example: state/role duration summary.
SELECT
  state_key,
  assigned_role,
  COUNT(*) AS completed_tasks,
  ROUND(AVG(EXTRACT(EPOCH FROM (completed_at - created_at)) / 3600)::numeric, 2) AS avg_hours,
  ROUND(PERCENTILE_CONT(0.95) WITHIN GROUP (
    ORDER BY EXTRACT(EPOCH FROM (completed_at - created_at)) / 3600
  )::numeric, 2) AS p95_hours
FROM workflow_task
WHERE status = 'completed'
  AND completed_at IS NOT NULL
GROUP BY state_key, assigned_role
ORDER BY p95_hours DESC;
```

12. Security, Governance, dan Auditability

Karena meeting mencatat concern KHS/security approval, keputusan stack dan library perlu dituangkan dalam Technology Decision Record. Selain approval stack, desain harus menghindari risiko spesifik sistem dinamis: arbitrary code execution dari metadata, XSS dari label/help text, upload file berbahaya, BPMN XML injection, dan broken access control antar entity [M1][M2].

Area Risiko	Kontrol yang Disarankan	Catatan
Authorization	RBAC + entity context + ownership check per endpoint.	Jangan hanya filter di frontend.
Audit	Append-only workflow_event; tidak boleh hard delete event.	Support forensic dan bottleneck.
Dynamic form XSS	Sanitize label/help text; render dengan component binding, bukan v-html.	Metadata dari admin tetap dianggap untrusted.
Expression/condition	Gunakan DSL/JSON rule terbatas, bukan eval/new Function.	Contoh: { field: "urgency", op: "eq", value: "High" }.
BPMN XML	Limit file size, validate XML, reject external entity, store checksum.	Jangan trust BPMN ID/name sebagai SQL identifier.
File upload	MIME whitelist, size limit, virus scan bila tersedia, object private by default.	Access via presigned URL time-limited.
Secrets	Credential DB/MinIO via secret manager/env, rotate access key.	Tidak di-commit ke repo.
Data retention	Retention policy attachment dan workflow event sesuai regulasi internal.	Definisikan sejak SRS.

13. Evaluasi Tech Stack

Stack awal cukup reasonable untuk prototype/MVP, tetapi perlu dikunci melalui TDR dan approval security. Catatan khusus: istilah "MinIO 9.0.1" perlu diklarifikasi karena MinIO server umumnya memakai release/tag date-based, sedangkan

versi 9.x sering merujuk ke SDK tertentu seperti Java SDK. Untuk backend Node.js, integrasi MinIO memakai package JavaScript SDK `minio` atau AWS S3 compatible SDK, bukan MinIO Java SDK [R7][R8].

Komponen	Versi Awal	Assessment	Rekomendasi
Vue.js	3.5.35	Layak untuk SPA dan dynamic form renderer; Vue 3 mendukung SFC dan Composition API untuk aplikasi full [R16].	Gunakan TypeScript, Vite, Pinia, Vue Router, reusable field components.
Node.js	24.16.0	Node.js 24 tercatat sebagai LTS; v24.16.0 adalah release LTS resmi Mei 2026 [R1][R2].	Gunakan TypeScript strict; freeze major ke Node 24 LTS untuk production.
Backend Framework	Belum ditentukan	Express minimal; Fastify cepat; NestJS memberi struktur modular enterprise [R15].	Rekomendasi: NestJS + Fastify adapter atau Fastify modular jika tim kuat TypeScript.
MongoDB	8.3.2	MongoDB docs menampilkan 8.3-series sebagai current stable; minor releases dinyatakan suitable for production workloads [R3][R4].	Gunakan untuk dynamic form bila approved; buat schema validation dan indexes.
PostgreSQL	16	Cocok untuk workflow transaction, audit, RBAC, report; JSONB tersedia jika perlu fleksibilitas [R6].	Jadikan source of truth workflow dan reporting.
MinIO	9.0.1?	Perlu klarifikasi apakah server atau SDK. MinIO JS SDK tersedia untuk akses S3-compatible object storage [R7][R8].	Gunakan object storage private, presigned URL, bucket policy ketat.
BPMN Library	Belum ditentukan	bpmn-js untuk browser viewer/modeler; bpmn-moddle untuk read/write BPMN XML di JS [R10][R11].	Gunakan bpmn-js frontend, bpmn-moddle backend parser; batasi subset MVP.
Form Library	Belum ditentukan	JSON Forms support Vue; FormKit schema mendukung form generation [R12][R13].	PoC dua opsi 2-3 hari, pilih yang paling cocok UI dan kontrol metadata.

14. Ide Implementasi Code

Bagian ini berisi skeleton ide code untuk mengarahkan developer. Kode bersifat konseptual dan perlu disesuaikan dengan framework final, ORM/query builder, naming convention, error handling, logging, dan standar keamanan proyek.

Project structure idea

```
// Recommended folder structure for modular monolith.
src/
  main.ts
  app.module.ts
  modules/
    auth/
    organization/
    probis/
      probis.controller.ts
      probis.service.ts
      probis.repository.ts
      dto/
    bpmn/
      bpmn-import.controller.ts
      bpmn-parser.service.ts
      bpmn-artifact.repository.ts
    workflow/
      workflow-definition.controller.ts
      workflow-instance.controller.ts
      workflow-transition.service.ts
      workflow-task.repository.ts
      workflow-event.repository.ts
      guards/
      transition-rules/
    forms/
      form-template.controller.ts
      form-render-schema.service.ts
      form-submission.service.ts
      validators/
    reporting/
      bottleneck-report.service.ts
    object-storage/
      minio.service.ts
```

```
  shared/
    database/
    errors/
    logger/
    audit/
```

Safe guard DSL idea

```
// Guard DSL idea: safe condition, not eval.
type GuardRule =
  | { type: 'FORM_FIELD_EQUALS'; formCode: string; field: string; value: string | number | boolean }
  | { type: 'USER_HAS_ROLE'; role: string }
  | { type: 'ENTITY_MATCHES_INSTANCE'; field: 'ownerEntityId' | 'requesterEntityId' }
  | { type: 'ALWAYS' };

async function evaluateGuard(rule: GuardRule, ctx: GuardContext): Promise<boolean> {
  switch (rule.type) {
    case 'ALWAYS': return true;
    case 'USER_HAS_ROLE': return ctx.user.roles.includes(rule.role);
    case 'FORM_FIELD_EQUALS': {
      const submission = await ctx.forms.getLatest(ctx.instance.id, rule.formCode);
      return submission?.data?.[rule.field] === rule.value;
    }
    case 'ENTITY_MATCHES_INSTANCE':
      return ctx.user.activeEntityId === ctx.instance[rule.field];
    default:
      assertNever(rule);
  }
}
```

MinIO service idea

```
// MinIO service idea: store files privately, return presigned URLs only when authorized.
import * as Minio from 'minio';

export class ObjectStorageService {
  private client = new Minio.Client({
    endPoint: process.env.MINIO_ENDPOINT!,
    port: Number(process.env.MINIO_PORT ?? 9000),
    useSSL: process.env.MINIO_SSL === 'true',
    accessKey: process.env.MINIO_ACCESS_KEY!,
    secretKey: process.env.MINIO_SECRET_KEY!,
  });

  async putWorkflowAttachment(params: {
    buffer: Buffer; bucket: string; objectKey: string; contentType: string; metadata: Record<string,string>;
  }) {
    await this.client.putObject(params.bucket, params.objectKey, params.buffer, params.buffer.length, {
      'Content-Type': params.contentType,
      ...params.metadata,
    });
    return { bucket: params.bucket, objectKey: params.objectKey };
  }

  async getPresignedReadUrl(bucket: string, objectKey: string, expirySeconds = 300) {
    return this.client.presignedGetObject(bucket, objectKey, expirySeconds);
  }
}
```

15. Roadmap 3 Bulan

Roadmap berikut dibuat agar stakeholder melihat demo bertahap, sesuai arah meeting yang meminta milestone quick win per sprint [M1][M2]. Asumsi sprint 2 minggu, total 6 sprint.

Sprint	Fokus	Output Demo	Acceptance Criteria
Sprint 0 / Inception	Scope lock, SRS draft, TDR, data model awal, backlog.	Scope baseline, architecture draft, PoC plan.	Scope in/out disetujui; stack approval path jelas.
Sprint 1	Probis Collection dan master/entity.	CRUD Probis, entity mapping, step list, version draft.	User dapat membuat Probis dan mapping entity.
Sprint 2	BPMN import/render dan parser dasar.	Upload BPMN, render diagram, extract task/flow.	Minimal satu BPMN sample terbaca dan tersimpan sebagai metadata.
Sprint 3	Form Builder metadata dan generated form.	Create form template, bind ke state, render form, submit data.	Field text/select/date/number/file berjalan dengan validation.
Sprint 4	Workflow State Engine.	Start instance, task list, approve, return, reject, audit.	Satu scenario end-to-end dengan history lengkap.
Sprint 5	Dashboard/reporting, hardening, SIT/UAT prep.	Bottleneck report, aging task, UAT script, deployment package.	UAT scenario utama pass; issue critical ditutup.

16. Testing Strategy

Layer	Test	Contoh
Unit	Transition resolver, guard evaluator, form validator, BPMN parser.	Given current_state=review, action=APPROVE -> to_state=approval.
Integration	DB transaction, MinIO upload, Mongo submission, API auth.	Transition creates event + closes old task + creates next task in one transaction.
Contract	OpenAPI schema dan response shape.	Frontend form submit sesuai backend validation error format.
Security	RBAC/entity access, upload MIME/size, XML hardening, XSS.	User entity A tidak bisa read task entity B.
Performance	List task/report pagination, upload file, bottleneck query.	Index task status/state/org; report p95 query < target.
UAT	Business scenario end-to-end.	Probis -> workflow -> form state 1/3/4 -> approve/return -> report.

17. Risk Register

Risk	Level	Trigger	Mitigation
Scope melebar menjadi low-code/SAP-like platform.	High	Stakeholder meminta semua form/layout/workflow dapat dibuat bebas.	Tuliskan non-goals; fokus 1-2 scenario; demo MVP per sprint.
Full BPMN execution dianggap wajib.	High	BPMN dengan gateway/event kompleks harus auto-run.	Tetapkan BPMN compatible subset; complex element manual mapping/phase 2.
Generated form dianggap kurang bagus.	High	User membandingkan dengan custom UI Figma.	Acceptance criteria UI sejak awal: rapi, usable, Google Form-like.
Approval Node/MongoDB/MinIO belum final.	High	KHS/security belum menyetujui stack.	Buat TDR; siapkan fallback PostgreSQL JSONB dan Java/Spring adapter bila perlu.
Workflow audit tidak konsisten.	High	Update state tanpa event atau event tidak transactional.	Semua transition via service tunggal dan DB transaction.
Perubahan form merusak data lama.	Medium	Template diedit tanpa versioning.	Publish version baru; submission simpan template snapshot.
Reporting lambat.	Medium	Event besar tanpa index/materialized view.	Index, materialized view/report table, async aggregation.
Multi-database operational burden.	Medium	PostgreSQL + MongoDB + MinIO butuh backup/monitoring berbeda.	Gunakan Mongo hanya bila approved; fallback JSONB.

18. Keputusan Teknis yang Perlu Dikunci

Decision	Opsi	Rekomendasi SA	Deadline
Backend structure	NestJS, Fastify direct, Express.	NestJS + Fastify adapter jika butuh struktur enterprise; Fastify direct jika tim ingin minimal.	Sprint 0
Form schema approach	JSON Forms, FormKit Schema, custom metadata.	PoC 2-3 hari; untuk kontrol tinggi, custom metadata sederhana + backend validator.	Sprint 1
Form storage	MongoDB vs PostgreSQL JSONB.	MongoDB bila approved; fallback PostgreSQL JSONB.	Sprint 0

Decision	Opsi	Rekomendasi SA	Deadline
BPMN execution level	Visual only, import metadata, full execution.	Import metadata + custom state machine. Full execution out-of-scope MVP.	Sprint 0
Return/reject rules	Previous, initiator, specific state, final reject.	Support previous + initiator + final reject MVP; specific state controlled by rule.	Sprint 2
Definition versioning	Mutable vs immutable published version.	Published version immutable; new change = new version.	Sprint 1
Security approval	Node/Mongo/MinIO/libs approved or not.	TDR + written approval / fallback path.	Before build

19. Kesimpulan dan Next Action

Secara SA, arah paling kuat adalah membangun platform metadata-driven dengan batasan MVP tegas. Probis Collection menjadi fondasi knowledge base, BPMN menjadi visual dan sumber metadata, Workflow Engine menjadi custom state machine yang audit-first, dan Form Builder menjadi generated form sederhana yang versioned. Dengan desain ini, project tetap memenuhi requirement utama: compatible BPMN, tracking tahap, approval, reject/return, form per state, dan bottleneck analysis, tanpa terjebak membangun full low-code platform dalam 3 bulan.

Priority	Action	Owner yang Disarankan	Output
P0	Kunci scope MVP dan non-goals.	PM/BA/SA/Stakeholder	Scope baseline dan acceptance criteria.
P0	Buat Technology Decision Record.	SA/Tech Lead	Keputusan Node, MongoDB/PostgreSQL JSONB, MinIO, BPMN/form library.
P0	PoC BPMN parser dan form renderer.	Tech Lead/Developer	BPMN sample dan form sample bisa didemokan.
P0	Finalisasi conceptual data model.	SA/DBA/Tech Lead	ERD awal dan Mongo collection design.
P1	Buat workflow transition prototype.	Backend Developer	Submit/approve/return/reject + audit event.
P1	Buat dashboard bottleneck awal.	Backend/Frontend	Aging task, average/p95 per state/role.
P1	Siapkan UAT scenario end-to-end.	BA/QA/SA	Scenario Probis + Workflow + Form + Report.

20. Referensi

Internal meeting references:

ID	Reference
[M1]	Summary_Detail_ProbisCollection_WorkflowEngine_FormBuilder(1).docx - uploaded meeting summary, 4 Juni 2026.
[M2]	MoM_ProbisCollection_WorkflowEngine_FormBuilder(1).docx - uploaded minutes of meeting, 4 Juni 2026.

External/public references used for technical research:

ID	Title / Note	URL
[R1]	Node.js Releases - status table showing Node.js v24 Krypton as LTS.	https://nodejs.org/en/about/previous-releases
[R2]	Node.js 24.16.0 (LTS) release notes.	https://nodejs.org/en/blog/release/v24.16.0
[R3]	MongoDB Release Notes - current stable release 8.3-series.	https://www.mongodb.com/docs/manual/release-notes/
[R4]	MongoDB Versioning - minor releases stable and suitable for production workloads.	https://www.mongodb.com/docs/manual/reference/versioning/
[R5]	MongoDB Node.js Driver release notes.	https://www.mongodb.com/docs/drivers/node/current/reference/release-notes/
[R6]	PostgreSQL 16 Documentation - JSON types and jsonb indexing.	https://www.postgresql.org/docs/16/datatype-json.html

ID	Title / Note	URL
[R7]	MinIO SDK documentation.	https://docs.min.io/aistor/developers/sdk/
[R8]	MinIO JavaScript Client API Reference.	https://docs.min.io/aistor/developers/sdk/javascript/api/
[R9]	OMG BPMN 2.0.2 specification overview.	https://www.omg.org/spec/BPMN/2.0.2/About-BPMN
[R10]	bpmn-js toolkit documentation.	https://bpmn.io/toolkit/bpmn-js/
[R11]	bpmn-moddle repository - read/write BPMN 2.0 XML from JavaScript.	https://github.com/bpmn-io/bpmn-moddle
[R12]	JSON Forms - JSON Schema based forms with Vue support.	https://jsonforms.io/
[R13]	FormKit schema documentation.	https://formkit.com/getting-started/what-is-formkit
[R14]	XState documentation - state management and orchestration for JavaScript/TypeScript.	https://stately.ai/docs/xstate
[R15]	NestJS official site - scalable server-side applications.	https://nestjs.com/
[R16]	Vue.js official guide - Vue 3 introduction and SFC/Composition API.	https://vuejs.org/guide/introduction.html

Disclaimer: Dokumen ini merupakan technical recommendation dan perlu divalidasi kembali terhadap standar internal, KHS/security approval, ketersediaan environment, serta requirement final stakeholder. Code snippets bersifat ide awal, bukan final production code.